Enseignement supérieur pour adultes
Rue Gatti de Gamond 95
1180 Uccle Belgique

Initiation aux bases de données
(M.Belhadj, E.Derasse)

Projet d'implémentation d'une base de données

Cyrille FLAMANT

Table des matières

Introduction 2
Modèle conceptuel de données 2
Modèle logique de données 3
Respect de la normalité 3
Création de la base de données 3
Exemple de requête de création : table Affectation 3
Exemple de requête d'insertion : données de la table TableResto 4
Requêtes SQL personnelles 4
Requête avec l'opérateur « IN » 4
Requête utilisant une fonction d'agrégation 4
Requête avec opérateur de groupement 5
Requête utilisant une vue 5

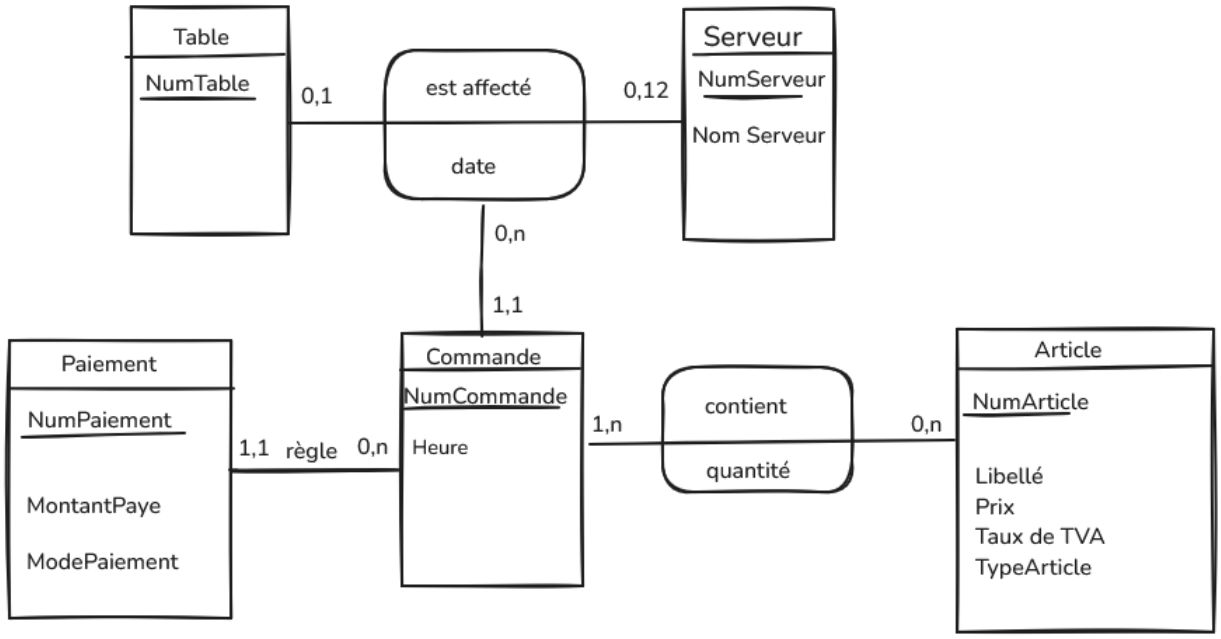
Introduction

L’objectif de ce projet est de développer une base de données pour un restaurant permettant la génération d’un ticket de caisse.
Les données du ticket doivent permettre d’identifier le serveur, la date de la commande, le n° de table, le ou les modes de paiement ainsi que le prix des articles et le prix total HT et TTC.

L’application proposée est structurée comme suit :
Les données des tables « Serveur », « Table » et « Article » sont enregistrées durablement dans la base de données.
Chaque jour, on affecte les serveurs actifs à plusieurs tables. Pour cela, on crée des enregistrements Date/Serveur/Table dans la table Affectation.
A chaque commande, le serveur introduit le numéro de table. Un enregistrement est alors créé dans la table Commande avec le numéro d’affectation.
Le serveur introduit également le nom des articles commandés et leur quantité, qui sont enregistrés dans la table LigneCommande avec le numéro de commande associé.
Lors du paiement, le montant total peut être réparti sur plusieurs paiements qui sont enregistrés dans la table Paiement et affectés au numéro de commande correspondant.
Des données simulant l'affectation des serveurs ainsi que la prise de 4 commandes et leur paiement ont été insérées.

Pour la génération du ticket de caisse, un script python permet de saisir un numéro de commande et interroge la base de données pour générer un ticket de caisse.

Modèle conceptuel de données



Modèle logique de données

Serveur (**NumServeur**, NomServeur)

TableResto (**NumTable**)

Affectation (**NumAffectation**, # NumServeur, # NumTable, DateAffectation)
UNIQUE (NumTable, DateAffectation)

Commande (**NumCommande**, # NumAffectation, Heure)

Article (**NumArticle**, Libellé, Prix, TypeArticle, TauxTVA)

LigneCommande (**# NumArticle, # NumCommande**, Quantité)

Paiement (**NumPaiement**, # NumCommande, MontantPaye, ModePaiement)

Respect de la normalité

- 1ère loi normale : OK, tous les attributs sont atomiques et non répétitifs

- 2ème loi normale : OK, la 1ère loi normale est respectée et aucun attribut ne dépend d'une partie d'une clé primaire

- 3ème loi normale : OK, la 2ème loi normale est respectée et aucune dépendance transitive n'est présente. La table Affectation peut poser question, car elle clé contient deux clés "candidates" : la clé primaire artificielle NumAffectation et la clé composée (#NumServeur, #NumTable, DateAffectation). Néanmoins, il ne s'agit pas ici d'une dépendance transitive, mais de la coexistence d'une clé technique et d'une clé "métier".

Création de la base de données

Exemple de requête de création : table Affectation

```
CREATE TABLE IF NOT EXISTS Affectation (
NumAffectation INTEGER PRIMARY KEY,
NumServeur INTEGER NOT NULL REFERENCES Serveur(NumServeur),
NumTable INTEGER NOT NULL REFERENCES TableResto(NumTable),
DateAffectation TEXT NOT NULL,
UNIQUE (NumTable, DateAffectation)
) ;
```

Explication :

La commande CREATE TABLE IF NOT EXISTS crée une table uniquement si une table du même nom n'existe pas déjà.

On affecte ensuite à la table l'attribut NumAffectation, un entier (INTEGER) que l'on définit comme clé primaire (PRIMARY KEY).
On définit ensuite les deux clés secondaires : NumServeur et NumTable, qui font référence aux attributs du même nom dans les tables Serveur et TableResto. « NOT NULL » indique qu'on ne peut pas créer d'enregistrements en laissant ce champ vide.
L'attribut « DateAffectation » est au format TEXT car il n'existe pas de format spécifique « date » en SQLite.
Enfin, la contrainte « UNIQUE » indique que chaque ensemble d'attributs NumTable/Date doit être unique ; ceci évite l'affectation d'une table à plusieurs serveurs à la même date.

Exemple de requête d'insertion : données de la table TableResto

```
INSERT INTO TableResto (NumTable, NumServeur) VALUES
(1,1),(2,1),(3,1),(4,1),
(5,2),(6,2),(7,2),(8,2),
(9,3),(10,3),(11,3),(12,3)
;
```

Explication :

La commande INSERT INTO est suivie du nom de la table, puis entre parenthèses du nom des attributs auxquels on souhaite affecter des valeurs.
Chaque enregistrement est entré entre parenthèses après VALUES, en respectant l'ordre des attributs indiqué après INSERT INTO.

Requêtes SQL personnelles

Requête avec l'opérateur « IN »

Cette requête permet de calculer le total des paiements effectués par carte le jour même :

```
SELECT SUM (P.MontantPaye) /100.0 as TotalCarte
FROM Paiement P, Commande C, Affectation A
WHERE
P.NumCommande = C.NumCommande
AND
C.NumAffectation = A.NumAffectation
AND
P.ModePaiement IN ("Visa", "Bancontact", "Mastercard")
AND
A.DateAffectation = date()
;
```

Requête utilisant une fonction d'agrégation

Cette requête renvoie le nombre de commandes passées le jour même :

```
SELECT COUNT (C.NumCommande) as NbCommandes
FROM Commande C, Affectation A
WHERE
C. NumAffectation = A.NumAffectation
```

```
AND
A.DateAffectation = date()
;
```

Requête avec opérateur de groupement

Cette requête renvoie le montant des commandes traitées par chacun des serveurs le jour même, trié par ordre croissant.

```
SELECT SUM (P.MontantPaye) as TotalServeur, S.NomServeur
FROM Paiement P, Commande C, Serveur S, Affectation A
WHERE
P.NumCommande = C.NumCommande
AND
C.NumAffectation = A.NumAffectation
AND
A.NumServeur = S.NumServeur
GROUP BY S.NomServeur
ORDER BY TotalServeur
;
```

Requête utilisant une vue

Cette requête renvoie la quantité commandée de chaque article sur la journée, par ordre croissant de quantité.

```
CREATE VIEW QuantiteArticles AS
SELECT SUM(L.Quantite) AS NbArticles, Art.Libelle
FROM Article Art, Commande C, Affectation A, LigneCommande L
WHERE
L.NumArticle = Art. NumArticle
AND
L.NumCommande = C.NumCommande
AND
C.NumAffectation = A.NumAffectation
GROUP BY Art.Libelle
ORDER BY NbArticles
;
```